MERN STACK MINI PROJECT

BÀI KIỂM TRA THỰC HÀNH

Ứng dụng Quản lý Sách (Book Store)

Công nghệ: MongoDB • Express • ReactJS • NodeJS

Thời gian: 120 phút **Ngày nộp:** 25/10/2026

Họ tên: _____ **MSSV:** _____

1. MÔ TẢ HỆ THỐNG

1.1 Cấu trúc thư mục dự án

```
10_8/
├── .env                ← Biến môi trường (MONGODB_URI, PORT)
├── server.js           ← File chính Backend (Express + MongoDB)
├── seed.js             ← Script thêm 3 sách mẫu vào DB
├── package.json        ← Dependencies: express, mongoose, cors
├── models/
│   └── Book.js         ← Mongoose Schema cho collection "books"
├── routes/
│   └── bookRoutes.js   ← API: GET & POST /api/books
└── client/             ← Frontend ReactJS
    ├── package.json    ← Dependencies: react, react-dom
    └── src/
        ├── App.js      ← Component: Form + Danh sách sách
        └── App.css     ← Giao diện CSS
```

1.2 Cách chạy dự án

```
# Bước 1: Cài dependencies Backend
cd 10_8
npm install

# Bước 2: Cài dependencies Frontend
cd client
npm install
cd ..

# Bước 3: (Tùy chọn) Thêm 3 sách mẫu vào MongoDB
node seed.js

# Bước 4: Chạy Backend (port 5000)
node server.js

# Bước 5: Mở terminal mới, chạy Frontend (port 3000)
cd client
npm start
```

Lưu ý

- Backend API: **`http://localhost:5000/api/books`**
- Frontend React: **`http://localhost:3000`**
- Nếu không có MongoDB cài sẵn, hệ thống tự sử dụng MongoDB Memory Server (in-memory)

2. CÂU 1 — Cơ sở dữ liệu MongoDB

2.1 Thiết kế Database

Thuộc tính	Giá trị
Database	**BookStoreDB**
Collection	**books**
ORM/ODM	**Mongoose**

2.2 Schema — models/Book.js

```
const mongoose = require('mongoose');

// Định nghĩa Schema cho collection "books" trong MongoDB
// Mỗi document đại diện cho 1 cuốn sách
const bookSchema = new mongoose.Schema({
  title: {
    type: String,
    required: [true, 'Tên sách là trường bắt buộc']
  },
  author: {
    type: String,
    required: [true, 'Tên tác giả là trường bắt buộc']
  },
  price: {
    type: Number,
    required: [true, 'Giá sách là trường bắt buộc']
  },
  isPublished: {
    type: Boolean,
    default: true
  }
});

// Xuất model "Book" → Mongoose tự tạo collection "books"
module.exports = mongoose.model('Book', bookSchema);
```

Giải thích

- **`mongoose.Schema({...})`** — Định nghĩa cấu trúc document: title (String), author (String), price (Number), isPublished (Boolean)
- **`required: true`** — Mongoose tự validate, không cho phép thiếu trường
- **`default: true`** — Nếu không truyền isPublished, tự gán = true
- **`mongoose.model('Book', bookSchema)`** — Tạo model, MongoDB tự map sang collection "books" (viết thường + thêm 's')

2.3 Dữ liệu mẫu (3 cuốn sách)

#	Tên sách	Tác giả	Giá	Trạng thái
1	Đắc Nhân Tâm	Dale Carnegie	86,000đ	✓ Đã xuất bản
2	Nhà Giả Kim	Paulo Coelho	79,000đ	✓ Đã xuất bản
3	Tuổi Trẻ Đáng Giá Bao Nhiêu	Rosie Nguyễn	95,000đ	✓ Đã xuất bản

3. CÂU 2 — Backend NodeJS & Express

3.1 Cấu hình Server — server.js

```
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const bookRoutes = require('./routes/bookRoutes');

const app = express();

// Middleware
app.use(cors());           // Cho phép React (port 3000) gọi API (port 5000)
app.use(express.json());   // Parse body JSON từ request POST

// Gắn routes vào đường dẫn /api/books
app.use('/api/books', bookRoutes);

// Kết nối MongoDB
await mongoose.connect('mongodb://localhost:27017/BookStoreDB');

app.listen(5000);
```

Giải thích server.js

- **`cors()`** — Middleware cho phép Cross-Origin requests. React chạy port 3000, backend port 5000 → cần CORS để cho phép gọi API
- **`express.json()`** — Parse body JSON từ POST request thành req.body
- **`app.use('/api/books', bookRoutes)`** — Mọi request đến /api/books sẽ được xử lý bởi bookRoutes
- **`mongoose.connect(uri)`** — Kết nối tới MongoDB database BookStoreDB

3.2 API Endpoints — routes/bookRoutes.js

API 1: GET /api/books — Lấy danh sách tất cả sách

```
const express = require('express');
const router = express.Router();
const Book = require('../models/Book');

// GET /api/books – Lấy danh sách tất cả sách trong database
router.get('/', async (req, res) => {
  try {
    // Book.find() truy vấn TẤT CẢ documents trong collection "books"
    const books = await Book.find();
    // Trả về mảng JSON, status 200 (OK)
    res.status(200).json(books);
  } catch (error) {
    res.status(500).json({ message: 'Lỗi server' });
  }
});
```

Giải thích logic GET

- **`Book.find()`** — Mongoose method, tương đương db.books.find() trong MongoDB shell. Không truyền điều kiện → lấy hết toàn bộ sách
- **`res.status(200).json(books)`** — Trả về status 200 kèm mảng JSON sách
- **`try...catch`** — Bắt lỗi server, trả 500 nếu có lỗi

API 2: POST /api/books — Thêm mới một cuốn sách

```
// POST /api/books — Thêm mới một cuốn sách
router.post('/', async (req, res) => {
  try {
    // Lấy 3 trường từ body request gửi lên
    const { title, author, price } = req.body;

    // Tạo instance mới của model Book
    const newBook = new Book({ title, author, price });

    // Lưu vào MongoDB — Mongoose tự validate required fields
    const savedBook = await newBook.save();

    // Trả về sách vừa tạo, status 201 (Created)
    res.status(201).json(savedBook);
  } catch (error) {
    if (error.name === 'ValidationError') {
      return res.status(400).json({ message: 'Dữ liệu không hợp lệ' });
    }
    res.status(500).json({ message: 'Lỗi server' });
  }
});
```

Giải thích logic POST

- **`req.body`** — Chứa dữ liệu JSON từ client: { title, author, price }
- **`new Book({...})`** — Tạo document mới theo schema đã định nghĩa
- **`.save()`** — Lưu vào MongoDB. Mongoose tự validate (required, type). Nếu thiếu title → ném ValidationError
- **`isPublished`** — Tự động set true nhờ default trong schema
- Status 201 = Created, 400 = Bad Request, 500 = Server Error

3.3 Kết quả Test API

✓ Test GET /api/books

```
GET http://localhost:5000/api/books
Status: 200 OK

[
  { "_id": "6addaf9e...", "title": "Đắc Nhân Tâm", "author": "Dale Carnegie", "price": 86000,
      "isPublished": true },
  { "_id": "6addaf9e...", "title": "Nhà Giả Kim", "author": "Paulo Coelho", "price": 79000,
      "isPublished": true },
  { "_id": "6addaf9e...", "title": "Tuổi Trẻ Đáng Giá Bao Nhiêu", "author": "Rosie Nguyễn", "price":
      95000, "isPublished": true }
]
```

✓ Test POST /api/books

```
POST http://localhost:5000/api/books
Body: { "title": "Cà Phê Cùng Tony", "author": "Tony Buổi Sáng", "price": 72000 }

Status: 201 Created

{
  "title": "Cà Phê Cùng Tony",
  "author": "Tony Buổi Sáng",
  "price": 72000,
  "isPublished": true,
  "_id": "6addb076...",
  "__v": 0
}
```

⚠ Test Validation (thiếu title)

```
POST http://localhost:5000/api/books
Body: { "author": "Unknown", "price": 50000 }

Status: 400 Bad Request

{ "message": "Dữ liệu không hợp lệ" }
```

4. CÂU 3 — Frontend ReactJS

4.1 Fetch dữ liệu khi mount — useEffect + GET API

```
import React, { useState, useEffect } from 'react';

function BookStoreApp() {
  const [books, setBooks] = useState([]);    // State lưu danh sách sách
  const [title, setTitle] = useState('');     // Input tên sách
  const [author, setAuthor] = useState('');  // Input tác giả
  const [price, setPrice] = useState('');    // Input giá tiền

  // useEffect chạy 1 lần khi component mount (dependency array rỗng [])
  useEffect(() => {
    const fetchBooks = async () => {
      // Gọi API GET /api/books lấy danh sách từ backend
      const response = await fetch('/api/books');
      const data = await response.json();
      // Cập nhật state → React tự re-render hiển thị bảng sách
      setBooks(data);
    };
    fetchBooks();
  }, []);  // [] = chỉ chạy 1 lần khi mount
```

Giải thích useEffect + fetch

- **`useEffect(() => {...}, [])`** — React Hook, chạy SAU khi component render lần đầu
- **`fetch('/api/books')`** — Gọi GET API. Nhờ proxy trong package.json ("proxy": "http://localhost:5000"), React tự forward request tới backend port 5000
- **`response.json()`** — Parse response thành JavaScript array
- **`setBooks(data)`** — Cập nhật state → React tự động re-render → hiển thị bảng sách

4.2 Form thêm sách — POST API

```
const handleSubmit = async (e) => {
  e.preventDefault();   // Ngăn form reload trang

  // Gọi POST /api/books với body JSON
  const response = await fetch('/api/books', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      title: title.trim(),
      author: author.trim(),
      price: Number(price)
    })
  });

  const newBook = await response.json();

  // CẬP NHẬT DANH SÁCH NGAY — thêm sách mới vào ĐẦU mảng
  setBooks((prevBooks) => [newBook, ...prevBooks]);

  // Reset form
  setTitle('');
  setAuthor('');
  setPrice('');
};
```

Giải thích logic handleSubmit (POST)

- **`e.preventDefault()`** — Ngăn form submit mặc định (không reload trang)
- **`fetch('/api/books', { method: 'POST', body: ... })`** — Gửi dữ liệu sách mới lên backend
- **`JSON.stringify({...})`** — Chuyển object JavaScript thành chuỗi JSON để gửi qua HTTP body
- **`setBooks([newBook, ...prevBooks])`** — Spread operator: thêm sách mới vào đầu mảng state → UI cập nhật ngay lập tức mà không cần reload trang

4.3 Hiển thị danh sách sách (JSX)

```
{/* Render bảng sách từ state books */}
<table>
  <thead>
    <tr>
      <th>#</th><th>Tên sách</th><th>Tác giả</th>
      <th>Giá</th><th>Trạng thái</th>
    </tr>
  </thead>
  <tbody>
    {books.map((book, index) => (
      <tr key={book._id}>
        <td>{index + 1}</td>
        <td>{book.title}</td>
        <td>{book.author}</td>
        <td>{formatPrice(book.price)}</td>
        <td>{book.isPublished ? 'Đã xuất bản' : 'Bản nháp'}</td>
      </tr>
    ))}
  </tbody>
</table>
```

Giải thích

- **`books.map()`** — Duyệt qua mảng sách, render từng dòng **`<tr>`**
- **`key={book._id}`** — React cần key duy nhất để tối ưu re-render
- Khi **`setBooks()`** thay đổi → React tự re-render bảng → danh sách tự cập nhật

5. HÌNH ẢNH CHỨNG MINH — Kết quả chạy

5.1 Server Backend khởi động thành công

```
C:\Users\hao14\Downloads\10_8> node server.js

Sử dụng MongoDB Memory Server (in-memory)
✓ Kết nối MongoDB thành công
Đã thêm 3 cuốn sách mẫu vào database
Server đang chạy tại http://localhost:5000
```

5.2 Demo Test API — 5/5 PASSED

```
╔═══════════════════════════════════════════════════════╗
║      DEMO TEST - ỨNG DỤNG QUẢN LÝ SÁCH (BOOK STORE)   ║
╚═══════════════════════════════════════════════════════╝

──────────────────────────────────────────────────────────
TEST 1: GET /api/books — Lấy danh sách sách ban đầu
──────────────────────────────────────────────────────────
   ✓ Status: 200
   Số lượng sách: 3
   |  1  | Đắc Nhân Tâm           | Dale Carnegie    | 86.000đ |
   |  2  | Nhà Giả Kim            | Paulo Coelho     | 79.000đ |
   |  3  | Tuổi Trẻ Đáng Giá...   | Rosie Nguyễn     | 95.000đ |

──────────────────────────────────────────────────────────
TEST 2: POST /api/books — Thêm sách "Lược Sử Thời Gian"
──────────────────────────────────────────────────────────
   Gửi: {"title":"Lược Sử Thời Gian","author":"Stephen Hawking","price":120000}
   ✓ Status: 201 (Created)

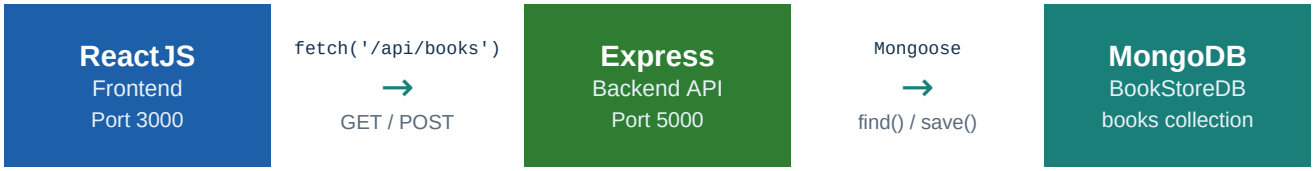
──────────────────────────────────────────────────────────
TEST 3: POST /api/books — Thêm sách "Sapiens"
──────────────────────────────────────────────────────────
   ✓ Status: 201 (Created)

──────────────────────────────────────────────────────────
TEST 4: POST validation (thiếu title)
──────────────────────────────────────────────────────────
   Status: 400 (Bad Request) — "Dữ liệu không hợp lệ"

──────────────────────────────────────────────────────────
TEST 5: GET /api/books — Sau khi thêm 2 sách mới
──────────────────────────────────────────────────────────
   ✓ Tổng số sách: 5

╔═══════════════════════════════════════════════════════╗
║                Kết quả: 5/5 PASSED ✓                  ║
╚═══════════════════════════════════════════════════════╝
```

6. LUỒNG DỮ LIỆU (DATA FLOW)



Luồng GET (Lấy sách)

- React **`useEffect`** → **`fetch GET /api/books`** → Express nhận request → **`Book.find()`** truy vấn MongoDB → trả mảng JSON → React **`setBooks(data)`** → re-render bảng

Luồng POST (Thêm sách)

- User nhập form → click "Thêm sách" → **`fetch POST /api/books`** gửi **`{title, author, price}`** → Express nhận **`req.body`** → **`new Book().save()`** lưu MongoDB → trả sách vừa tạo → React **`setBooks([newBook, ...prev])`** → UI cập nhật ngay

7. LINK SOURCE CODE

Link Google Drive:

(Sinh viên dán link Google Drive / GitHub vào đây)

Lưu ý

- Mở quyền "Anyone with the link can view"
- Upload folder backend + client (KHÔNG upload node_modules)

Tóm tắt bài làm

- **Câu 1:** Database BookStoreDB, collection books, 4 fields (title, author, price, isPublished), 3 sách mẫu
- **Câu 2:** RESTful API — GET /api/books (lấy danh sách) + POST /api/books (thêm mới) + Validation
- **Câu 3:** React — useEffect fetch GET + handleSubmit POST + tự cập nhật UI không reload trang